

Niveau 5 — Expert : TNP, matrices aléatoires, fonctions L et cryptographie

Session 6 — Frontières de la recherche autour de $\zeta(s)$ Méthode :
théorie avancée → formule → exploration numérique → code Python

Objectif de cette session

Ce niveau franchit la frontière entre apprentissage et recherche. Quatre thèmes de recherche active, tous connectés à $\zeta(s)$:

1. **TNP et formule explicite** — comment les zéros de ζ gouvernent les premiers
 2. **Matrices aléatoires et GUE** — pourquoi les zéros se comportent comme des valeurs propres
 3. **Fonctions L généralisées** — la famille dont ζ est le membre le plus simple
 4. **Cryptographie et PRNG** — applications concrètes de la distribution des premiers
-

PARTIE 1 — Théorème des nombres premiers et formule explicite

1.1 Théorème des nombres premiers (TNP)

$$\pi(x) \sim \frac{x}{\log x} \quad \text{quand } x \rightarrow \infty$$

Forme plus précise via le logarithme intégral $\text{Li}(x) = \int_2^x \frac{dt}{\log t}$:

$$\pi(x) = \text{Li}(x) + O\left(x e^{-c\sqrt{\log x}}\right)$$

Si HR est vraie : l'erreur descend à $O(\sqrt{x} \log x)$ — bien meilleure.

1.2 Formule explicite de Riemann

La connexion exacte entre $\pi(x)$ et les zéros de ζ est donnée par la **formule explicite** :

$$\psi(x) = x - \sum_{\rho} \frac{x^{\rho}}{\rho} - \frac{\zeta'(0)}{\zeta(0)} - \frac{1}{2} \log(1 - x^{-2})$$

où $\psi(x) = \sum_{p^k \leq x} \log p$ (fonction de Tchebyshev de seconde espèce) et la somme porte sur tous les zéros non triviaux ρ .

Interprétation : x est le terme principal (croissance lisse), et chaque zéro $\rho = \frac{1}{2} + it$ contribue une oscillation :

$$\frac{x^\rho}{\rho} = \frac{x^{1/2} e^{it \log x}}{\rho}$$

Les zéros "sculpte" la distribution des premiers par interférence de ces ondes.

1.3 Lien $\pi(x) \leftrightarrow \psi(x)$

$$\pi(x) = \frac{\psi(x)}{\log x} + \int_2^x \frac{\psi(t)}{t (\log t)^2} dt$$

1.4 Exemple à la main

Pour $x = 100$:

- $\pi(100) = 25$ (25 premiers ≤ 100)
- $\text{Li}(100) \approx 29,08$
- Erreur = $|25 - 29,08| = 4,08$

Le premier terme de correction vient de $\rho_1 = \frac{1}{2} + 14,13i$:

$$\frac{100^{1/2+14,13i}}{1/2 + 14,13i} = \frac{10 e^{14,13i \log 100}}{1/2 + 14,13i}$$

module $\approx 10/14,13 \approx 0,7$ — l'oscillation est de faible amplitude pour $x = 100$, mais les premières corrections somment pour corriger vers 25.

1.5 Code Python — formule explicite et reconstruction de $\psi(x)$

```
from mpmath import mp, mpf, mpc, log, sqrt, exp, pi, zeta, re, im, fabs, power
from sympy import primerange, factorint
import numpy as np
import matplotlib.pyplot as plt

mp.dps = 20

#  $\psi(x)$  exact — fonction de Tchebyshev
def psi_exact(x):
    """ $\psi(x) = \sum_{p^k \leq x} \log(p)$ """
    s = mpf(0)
    for p in primerange(2, int(x)+1):
        pk = p
        while pk <= x:
            s += log(p)
            pk *= p
    return float(s)

#  $\psi(x)$  approché par formule explicite (N premiers zéros)
zeros_t = [14.134725141734693, 21.022039638771555, 25.010857580145688,
            30.424876125859513, 32.935061587739189, 37.586178158825671,
```

```

40.918719012147495, 43.327073280914999, 48.005150881167159,
49.773832477672302]

def psi_explicite(x, zeros_t):
    """Approximation de  $\psi(x)$  via la formule explicite de Riemann."""
    x = mpf(x)
    S = x # terme principal
    for t in zeros_t:
        rho = mpc(0.5, t)
        S -= re(power(x, rho) / rho)
        rho_conj = mpc(0.5, -t)
        S -= re(power(x, rho_conj) / rho_conj)
    S -= log(2*pi) # correction constante approx.
    return float(S)

# Comparaison
print("Reconstruction de  $\psi(x)$  par formule explicite:\n")
print(f"{'x':>8}|{' $\psi$  exact':>12}|{' $\psi$  explicite':>12}|{'Erreur%':>10}")
print("-" * 50)
for x in [10, 20, 50, 100, 200]:
    p_ex = psi_exact(x)
    p_app = psi_explicite(x, zeros_t)
    err = abs(p_ex - p_app) / p_ex * 100 if p_ex > 0 else 0
    print(f"{'x':>8}|{'p_ex':>12.4f}|{'p_app':>12.4f}|{'err':>9.2f}%")

# Graphique
x_vals = np.linspace(2, 150, 600)
psi_ex_vals = [psi_exact(x) for x in x_vals]
psi_app_vals = [psi_explicite(x, zeros_t) for x in x_vals]
psi_lisse = [float(x) for x in x_vals] # terme principal seul

fig, axes = plt.subplots(2, 1, figsize=(12, 9))

ax1 = axes[0]
ax1.plot(x_vals, psi_ex_vals, 'k-', linewidth=1.5, label=' $\psi(x)$  exact')
ax1.plot(x_vals, psi_app_vals, 'r--', linewidth=1.5, label=f' $\psi(x)$  formule explicite ({len(zeros_t)} zéros)')
ax1.plot(x_vals, psi_lisse, 'b:', linewidth=1.5, alpha=0.7, label=' $\psi(x) \approx x$  (terme principal)')
for p in primerange(2, 151):
    ax1.axvline(p, color='gray', linewidth=0.3, alpha=0.4)
ax1.set_xlabel('x')
ax1.set_ylabel('ψ(x)')
ax1.set_title('Formule explicite de Riemann - reconstruction de ψ(x)\nChaque zéro de ζ ajoute une oscillation')
ax1.legend(fontsize=9)
ax1.grid(True, alpha=0.3)

ax2 = axes[1]
erreur = [a - e for a, e in zip(psi_app_vals, psi_ex_vals)]
ax2.plot(x_vals, erreur, 'purple', linewidth=1.2)
ax2.axhline(0, color='k', linewidth=0.8)
ax2.set_xlabel('x')
ax2.set_ylabel('ψ_explicite(x) - ψ_exact(x)')
ax2.set_title('Erreur de troncature due aux zéros non inclus dans la somme')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("niveau5_formule_explicite.png", dpi=150)
plt.show()
print("Graphique sauvegardé: niveau5_formule_explicite.png")

```

PARTIE 2 — Conjecture de Montgomery et matrices aléatoires (GUE)

2.1 Espacement entre les zéros

On normalise les zéros t_n par leur densité locale :

$$\tilde{t}_n = t_n \cdot \frac{\log(t_n/2\pi)}{2\pi}$$

L'espacement normalisé entre zéros consécutifs est :

$$\delta_n = \tilde{t}_{n+1} - \tilde{t}_n$$

2.2 Conjecture de Montgomery (1973)

Montgomery a conjecturé que la **corrélation de paires** des zéros de ζ , après normalisation, suit la même loi que celle des **valeurs propres** d'une matrice aléatoire tirée de l'**Ensemble Unitaire Gaussien (GUE)** :

$$R_2(\alpha) = 1 - \left(\frac{\sin(\pi\alpha)}{\pi\alpha} \right)^2$$

Connexion avec la physique : les niveaux d'énergie de systèmes quantiques chaotiques (boule de billard quantique, noyaux atomiques lourds) suivent la même statistique GUE.

Les zéros de ζ et l'énergie des noyaux atomiques obéissent à la même loi — un mystère profond à la frontière entre théorie des nombres et physique quantique.

2.3 Distribution de l'espacement — loi de Wigner-Dyson

Pour GUE, la distribution des espacements δ suit approximativement la **loi de Wigner** :

$$P(\delta) \approx \frac{32}{\pi^2} \delta^2 e^{-4\delta^2/\pi}$$

Caractéristique : **répulsion de niveaux** — la probabilité que deux zéros soient très proches tend vers 0 quand $\delta \rightarrow 0$ (contrairement à des points de Poisson indépendants).

2.4 Exemple à la main — calcul de quelques espacements

À partir des premiers zéros $t_1 = 14,13$, $t_2 = 21,02$, $t_3 = 25,01$:

$$\delta_1 = 21,02 - 14,13 = 6,89 \quad \delta_2 = 25,01 - 21,02 = 3,99$$

Espacement moyen pour $t \approx 20$: $\frac{2\pi}{\log(20/2\pi)} \approx 5,4$

Espacement normalisé : $\tilde{\delta}_1 \approx 6,89/5,4 \approx 1,28$

2.5 Code Python — statistiques GUE vs zéros de ζ

```
from mpmath import mp, mpf, pi, log
import numpy as np
from scipy.stats import gaussian_kde
import matplotlib.pyplot as plt

mp.dps = 15

# Zéros de  $\zeta$  – table de Odlyzko (premiers 1000 zéros approx. via mpmath)
from mpmath import zetazero
mp.dps = 15

print("Calcul des 200 premiers zéros de  $\zeta$ ...")
N_zeros = 200
zeros = [float(zetazero(n).imag) for n in range(1, N_zeros+1)]
print(f"t_1={zeros[0]:.6f}")
print(f"t_{N_zeros}={zeros[-1]:.6f}")

# Normalisation par la densité locale
def densite_locale(t):
    return float(log(t / (2*pi)) / (2*pi))

zeros_norm = [t * densite_locale(t) for t in zeros]
espacements = [zeros_norm[n+1] - zeros_norm[n] for n in range(len(zeros_norm)-1)]

# Distribution de Wigner-Dyson (GUE)
s = np.linspace(0, 4, 500)
wigner = (32/np.pi**2) * s**2 * np.exp(-4*s**2/np.pi)

# Distribution de Poisson (processus indépendant – ce que ce N'EST PAS)
poisson = np.exp(-s)

fig, axes = plt.subplots(1, 2, figsize=(13, 6))

# Histogramme des espacements vs loi de Wigner
ax1 = axes[0]
ax1.hist(espacements, bins=40, density=True, alpha=0.6, color='steelblue',
        label=f'Espacements des {N_zeros} premiers zéros de  $\zeta$ ')
ax1.plot(s, wigner, 'r-', linewidth=2.5, label='Loi de Wigner-Dyson (GUE)')
ax1.plot(s, poisson, 'g--', linewidth=2, label='Loi de Poisson (indépendant)')
ax1.set_xlabel('Espacement normalisé  $\delta$ ')
ax1.set_ylabel('Densité de probabilité')
ax1.set_title('Distribution des espacements entre zéros de  $\zeta$  \nvs loi GUE – conjecture de Montgomery')
ax1.legend(fontsize=9)
ax1.grid(True, alpha=0.3)
ax1.set_xlim(0, 4)

# Statistiques des espacements
ax2 = axes[1]
ax2.plot(range(1, len(espacements)+1), espacements, 'b.', markersize=3, alpha=0.5)
ax2.axhline(np.mean(espacements), color='r', linewidth=2,
        label=f'Moyenne={np.mean(espacements):.4f} (théorique  $\approx 1$ )')
ax2.axhline(1.0, color='g', linewidth=1.5, linestyle='--', label='Valeur théorique = 1')
ax2.set_xlabel('Indice n')
ax2.set_ylabel('Espacement normalisé  $\delta_n$ ')
ax2.set_title('Espacements consécutifs entre les zéros normalisés \n(répulsion de niveaux visible)')
ax2.legend(fontsize=9)
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig("niveau5_gue_montgomery.png", dpi=150)
plt.show()
print("Graphique_sauvegardé:_niveau5_gue_montgomery.png")

# Statistiques de base
print(f"\nStatistiques_des_{len(espacements)}_espacements_normalisés:")
print(f"_{np.mean(espacements):.6f}_{théorique:_1.000}")
print(f"_{np.var(espacements):.6f}")
print(f"_{min(espacements):.6f}_{max(espacements):.6f}")
print(f"Nb_{0.1}_{sum(1_for_d_in_espacements_if_d<0.1)}_(répulsion_de_niveaux)")
```

PARTIE 3 — Fonctions L généralisées

3.1 Caractères de Dirichlet

Un **caractère de Dirichlet** modulo q est une fonction $\chi : \mathbb{Z} \rightarrow \mathbb{C}$ vérifiant :

- $\chi(n + q) = \chi(n)$ (périodicité)
- $\chi(mn) = \chi(m)\chi(n)$ (multiplicativité)
- $\chi(n) = 0$ si $\gcd(n, q) > 1$

Le caractère trivial χ_0 vaut 1 pour $\gcd(n, q) = 1$ et 0 sinon.

3.2 Fonctions L de Dirichlet

$$L(s, \chi) = \sum_{n=1}^{\infty} \frac{\chi(n)}{n^s} = \prod_p \frac{1}{1 - \chi(p) p^{-s}}$$

$\zeta(s)$ est le cas $\chi = \chi_0$ modulo 1.

Propriétés :

- $L(s, \chi)$ est holomorphe sur $\mathbb{C}$ si $\chi \neq \chi_0$
- $L(s, \chi_0)$ a un pôle simple en $s = 1$ (comme ζ)
- Équation fonctionnelle similaire à celle de ζ

3.3 Théorème de Dirichlet sur les premiers en progression arithmétique

Pour $\gcd(a, q) = 1$, il existe une infinité de premiers $p \equiv a \pmod{q}$.

La preuve utilise que $L(1, \chi) \neq 0$ pour tout $\chi \neq \chi_0$.

Densité : parmi les premiers $\leq x$, environ $\frac{\pi(x)}{\phi(q)}$ sont $\equiv a \pmod{q}$ (equirépartition), où ϕ est la fonction d'Euler.

3.4 Hypothèse de Riemann généralisée (GRH)

Tous les zéros non triviaux de $L(s, \chi)$ ont partie réelle $\frac{1}{2}$.

C'est l'analogue de HR pour toutes les fonctions L . La GRH implique des raffinements du TNP dans les progressions arithmétiques.

3.5 Code Python — calcul des fonctions L et vérification

```
from mpmath import mp, mpc, zeta, re, im, fabs, log, pi, exp
import numpy as np
import matplotlib.pyplot as plt

mp.dps = 15

# Caractère de Dirichlet  $\chi \bmod 4$  (le seul non trivial mod 4)
#  $\chi(1)=1, \chi(3)=-1, \chi(0)=\chi(2)=0$ 
def chi4(n):
    n = n % 4
    if n == 1: return 1
    if n == 3: return -1
    return 0

def L_dirichlet(s, chi, N=5000):
    """Calcul de  $L(s, \chi) = \sum \chi(n)/n^s$  par troncature."""
    s = mpc(s)
    total = mpc(0)
    for n in range(1, N+1):
        c = chi(n)
        if c != 0:
            total += mpc(c) / mpc(n)**s
    return total

# Comparaison  $\zeta(s)$  et  $L(s, \chi_4)$ 
print("Valeurs de  $\zeta(s)$  et  $L(s, \chi_4) = \sum (-1)^n / (2n+1)^s$  pour  $s$  réel : \n")
print(f"{'s':>6}|{' $\zeta(s)$ ':>14}|{' $L(s, \chi_4)$ ':>14}|Note")
print("-" * 58)
for s_val, note in [(1, 'pôle pour  $\zeta$ '), (2, ' $\pi^2/6$ '), (3, ''), (4, ' $\pi^4/90$ ')]:
    z = zeta(s_val) if s_val > 1 else float('inf')
    l = float(re(L_dirichlet(s_val, chi4)))
    print(f"{'s_val':>6}|{'float(z) if z != float('inf') else 'pôle':>14}|{'l':>14.8f}|{note}")

#  $L(1, \chi_4) = \pi/4$  (formule de Leibniz)
print(f"\n $L(1, \chi_4) = 1 - 1/3 + 1/5 - 1/7 + \dots = \pi/4 \approx \{{float(pi)/4:.8f}\}")$ ")
l1_num = float(re(L_dirichlet(1, chi4)))
print(f" $L(1, \chi_4)$  numérique =  $\{{l1_num:.8f}\}")$ ")

# Module  $|L(1/2 + it, \chi_4)|$  sur la droite critique
t_vals = np.linspace(1, 50, 500)
L_crit = [float(fabs(L_dirichlet(mpc(0.5, t), chi4, N=2000))) for t in t_vals]
Z_crit = [float(fabs(zeta(mpc(0.5, t)))) for t in t_vals]

fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(t_vals, Z_crit, 'b-', linewidth=1.5, alpha=0.8, label=' $|\zeta(1/2+it)|$ ')
ax.plot(t_vals, L_crit, 'r-', linewidth=1.5, alpha=0.8, label=' $|L(1/2+it, \chi_4)|$ ')
ax.axhline(0, color='k', linewidth=0.5)
ax.set_xlabel('t')
ax.set_ylabel('Module')
ax.set_title('  $|\zeta(1/2+it)|$  et  $|L(1/2+it, \chi_4)|$  sur la droite critique \n '
    'Les zéros des fonctions  $L$  généralisées obéissent à GRH (conjecturée)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("niveau5_fonctions_L.png", dpi=150)
plt.show()
print("Graphique sauvegardé : niveau5_fonctions_L.png")
```

PARTIE 4 — Cryptographie, PRNG ζ et connexions avancées

4.1 Distribution des premiers et cryptographie

La sécurité de **RSA** repose sur la difficulté de factoriser $N = p \cdot q$. Cette difficulté est liée à l'**irrégularité de la distribution des premiers**.

HR quantifie précisément cette irrégularité :

- HR vraie $\rightarrow$ erreur $O(\sqrt{x} \log x)$ dans le TNP $\rightarrow$ distribution "presque régulière"
- HR fausse $\rightarrow$ erreur bien plus grande $\rightarrow$ distribution bien plus irrégulière

Des algorithmes de génération de premiers (Miller-Rabin probabiliste, AKS déterministe) utilisent des résultats conditionnels à GRH.

4.2 Générateur pseudo-aléatoire basé sur ζ

Une idée expérimentale : utiliser les zéros de ζ comme source de bits pseudo-aléatoires.

Principe : les parties fractionnaires $\{t_n \cdot \log 2\}$ (où t_n sont les zéros) semblent se distribuer uniformément — propriété liée à la conjecture de Montgomery/GUE.

Ce PRNG est étudié dans la littérature comme test de l'hypothèse de normalité (Weyl, 1916) appliquée aux zéros.

4.3 Test de Turing — vérification algorithmique des zéros

Alan Turing a proposé en 1953 une méthode pour **certifier** qu'aucun zéro n'a été manqué dans un intervalle $[0, T]$:

Critère de Backlund : $N(T) = \frac{1}{\pi} \arg \zeta\left(\frac{1}{2} + iT\right) + \frac{T}{2\pi} \log \frac{T}{2\pi} - \frac{T}{2\pi} + \frac{7}{8} + O(1/T)$

Si le nombre de changements de signe de $Z(t)$ dans $[0, T]$ est égal à $N(T)$, **aucun zéro n'a été manqué** — ils sont tous simples et sur la droite critique.

4.4 Lean 4 — vers les preuves formelles

Mathlib4 (bibliothèque Lean 4) contient :

- La définition formelle de $\zeta(s)$
- La preuve du TNP
- Des lemmes sur $\Gamma(s)$

Exemple de structure d'une preuve Lean 4 autour de ζ :

```
-- Définition de  $\zeta$  pour  $\text{Re}(s) > 1$ 
noncomputable def riemannZeta :  $\mathbb{C} \rightarrow \mathbb{C} := \text{Complex.riemannZeta}$ 

-- Le TNP en Lean 4 (disponible dans Mathlib)
theorem prime_number_theorem :
  Tendsto (fun x =>  $\pi x / (x / \text{Real.log } x)$ ) atTop  $\square(1) := \text{by}$ 
```


L'objectif à long terme du projet : formaliser des résultats partiels autour de HR en utilisant Lean 4 + Mathlib4.

4.5 Code Python — PRNG ζ et test d'uniformité

```
from mpmath import mp, zetazero, frac, log, pi
import numpy as np
from scipy.stats import kstest, uniform
import matplotlib.pyplot as plt

mp.dps = 20

# Génération de la suite pseudo-aléatoire via les zéros de  $\zeta$ 
print("Calcul des 500 premiers zéros de  $\zeta$  pour le PRNG...")
N = 500
zeros_im = [float(zetazero(n).imag) for n in range(1, N+1)]

# Parties fractionnaires de  $t_n \cdot \log(2)$  – test de Weyl
seq_log2 = [float(frac(t * log(2))) for t in zeros_im]
# Parties fractionnaires de  $t_n / \pi(2)$  – espacement normalisé
seq_2pi = [float(frac(t / (2*pi))) for t in zeros_im]

# Test de Kolmogorov-Smirnov contre la loi uniforme [0,1]
ks_log2, p_log2 = kstest(seq_log2, 'uniform')
ks_2pi, p_2pi = kstest(seq_2pi, 'uniform')

print(f"\nTest de KS d'uniformité sur {N} valeurs:")
print(f"{{t_n . log 2}} : statistique KS = {ks_log2:.4f}, p-valeur = {p_log2:.4f}")
print(f"{{t_n / (2*pi)}} : statistique KS = {ks_2pi:.4f}, p-valeur = {p_2pi:.4f}")
print(f"({p_log2 > 0.05} pas de raison de rejeter l'uniformité)")

fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# Histogramme {t_n . log 2}
ax1 = axes[0][0]
ax1.hist(seq_log2, bins=25, density=True, alpha=0.7, color='steelblue')
ax1.axhline(1, color='r', linewidth=2, linestyle='--', label='Uniforme [0,1]')
ax1.set_title(f'Distribution de {{t_n . log 2}} \n KS={ks_log2:.4f}, p={p_log2:.3f}')
ax1.set_xlabel('Partie fractionnaire')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Diagramme de dispersion (t_n, t_{n+1}) pour détecter corrélations
ax2 = axes[0][1]
ax2.scatter(seq_log2[:-1], seq_log2[1:], s=3, alpha=0.4, color='steelblue')
ax2.set_title('Diagramme de retard (n, n+1) \n (absence de structure = bonne aléatoire)')
ax2.set_xlabel('n')
ax2.set_ylabel('n+1')
ax2.grid(True, alpha=0.3)

# Autocorrélation
ax3 = axes[1][0]
corr = np.correlate(seq_log2 - np.mean(seq_log2),
                    seq_log2 - np.mean(seq_log2), mode='full')
corr = corr[len(corr)//2:] / corr[len(corr)//2]
ax3.plot(range(50), corr[:50], 'b-o', markersize=4)
ax3.axhline(0, color='k', linewidth=0.8)
ax3.axhline(1.96/np.sqrt(N), color='r', linestyle='--', alpha=0.7, label='±1.96/N (intervalle 95%)')
```

```

ax3.axhline(-1.96/np.sqrt(N), color='r', linestyle='--', alpha=0.7)
ax3.set_title('Autocorrélation de {t_n·log_2}\n(doit rester dans les bandes rouges)')
ax3.set_xlabel('Décalage_k')
ax3.set_ylabel('Autocorrélation')
ax3.legend(fontsize=8)
ax3.grid(True, alpha=0.3)

# Évolution de la p-valeur KS en fonction de N
ax4 = axes[1][1]
ns = range(50, N, 10)
p_vals = [kstest(seq_log2[:n], 'uniform')[1] for n in ns]
ax4.plot(ns, p_vals, 'g-', linewidth=1.5)
ax4.axhline(0.05, color='r', linestyle='--', label='Seuil_p=0.05')
ax4.set_xlabel('Nombre de zéros utilisés(N)')
ax4.set_ylabel('p-valeur_KS')
ax4.set_title('Robustesse du test_KS selon N\n(p > 0.05 = uniformité non rejetée)')
ax4.legend()
ax4.grid(True, alpha=0.3)
ax4.set_ylim(0, 1)

plt.tight_layout()
plt.savefig("niveau5_prng_zeta.png", dpi=150)
plt.show()
print("Graphique sauvegardé : niveau5_prng_zeta.png")

```

Exercices de cette session

Exercice 1 — Formule explicite

Calculez la contribution du seul premier zéro $\rho_1 = \frac{1}{2} + 14,13i$ à la formule explicite pour $x = 50$, et estimez son poids relatif par rapport au terme principal x .

Exercice 2 — Statistiques GUE

Avec les 1000 premiers zéros (via zetazero), recalculez la distribution des espacements normalisés. Comparez la moyenne et la variance obtenues aux prédictions théoriques GUE : $\mathbb{E}[\delta] = 1$ et $\text{Var}[\delta] \approx 0,286$ (calculé numériquement pour GUE).

Exercice 3 — Fonctions L

Calculez $L(1, \chi_3)$ où χ_3 est le caractère de Dirichlet modulo 3 : $\chi_3(1) = 1$, $\chi_3(2) = -1$, $\chi_3(0) = 0$.

Comparez à la valeur exacte $L(1, \chi_3) = \pi/(3\sqrt{3}) \approx 0,6046 \dots$

Exercice 4 — Test de Turing

Implémentez le critère de Backlund pour vérifier qu'aucun zéro n'a été manqué dans l'intervalle $[0, 50]$: comptez les changements de signe de $Z(t)$ et comparez à $N(50)$ calculé par la formule de Weyl.

Carte mentale de cette session

(Diagramme Mermaid — version interactive sur le wiki.)

Résumé clé

Concept	Formule / Résultat	Importance
Formule explicite	$\psi(x) = x - \sum_{\rho} x^{\rho} / \rho + \dots$	Lien direct zéros $\leftrightarrow$ premiers
TNP conditionnel à HR	$\pi(x) = \text{Li}(x) + O(\sqrt{x} \log x)$	Meilleure borne connue sous HR
Conjecture Montgomery	Espacements $\sim$ GUE	Connexion théorie des nombres / phys
Loi de Wigner-Dyson	$P(\delta) \approx \frac{32}{\pi^2} \delta^2 e^{-4\delta^2/\pi}$	Répulsion de niveaux des zéros
Fonctions L	$L(s, \chi) = \sum \chi(n) / n^s$	Généralisation de ζ
GRH	$\text{Re}(\rho_{L, \chi}) = \frac{1}{2}$	HR pour toutes les fonctions L
PRNG ζ	$\{t_n \cdot \log 2\} \sim \mathcal{U}[0, 1]$	Aléatoire des zéros (conjectural)

Au-delà du niveau 5 — pistes de recherche

Thème	Description	Référen
Conjecture de Birch-Swinnerton-Dyer	Analogue de HR pour les courbes elliptiques	arXiv:m
Programme de Langlands	Unification des fonctions L	LMFDB,
Hypothèse de Lindelöf	$\$$	$\backslash \text{zeta}(1/2$
Moments de ζ	$\$ \int_0^T$	$\backslash \text{zeta}(1/2$
Zéros de ζ et primalité	AKS, Miller-Rabin sous GRH	Théorie

Dernière mise à jour : 22 mai 2026 — 585 lignes